A `parse_metadata` Függvény Sorról Sorra Elemzése

Ez a függvény felelős a projekt **értelmi magjáért**: a nyers OCR vagy digitális szövegből logikai szabályok (Regex), szótárak és adatbázis-összerendelések segítségével kinyeri a strukturált üzleti adatokat.

---

1. Függvényfejléc és Előkészítés

```
def parse_metadata(full_text: str, filename: str, invoice_lookup: dict) -> dict:
```

- **Szintaktika:**
  - `def`: Függvénydefiníció kezdete.
  - `full_text: str`: A PDF-ből kinyert teljes szövegtömeg.
  - `filename: str`: Az aktuálisan feldolgozott fájl neve (pl. `50643367_2026.04.27.pdf`).
  - `invoice_lookup: dict`: A DALI táblából (`podtabla.xlsx`) induláskor felépített szótár, ahol a kulcs a Shipment szám, az érték pedig a számlaszám.
  - `-> dict`: Típusjelölés: a függvény a kinyert adatokat egy Python szótárban (`dictionary`) adja vissza.
- **Logika:** A függvény megkap mindent, ami az azonosításhoz kell: a nyers szöveget, a fájl nevét és az SAP/DALI referenciatáblát.

```
# OCR hibák alapvető tisztítása (pl. betű O helyett szám 0)
text = full_text.replace("O", "0")
```

- **Szintaktika:** `string.replace(mit, mire)` metódus. Minden nagy `O` (O, mint Oszkár) karaktert kicserél `0` (nulla) számjegyre a szöveg másolatában.
- **Logika:**
  - Mivel a CMR dokumentumokon a kulcsszámok (Shipment: `2004...`, TO: `EL000...`, Dátum: `2026...`) szinte kizárólag számjegyekből állnak, az OCR leggyakoribb tévesztése, hogy a nullát `O` betűnek olvassa.
  - Ezzel az egyetlen cserével drasztikusan javul a számsorok illeszkedési aránya.

---

2. A Belső Kereső Segédfüggvény (`find_match`)

```
def find_match(patterns, text_block):
    for pattern in patterns:
        match = re.search(pattern, text_block, re.IGNORECASE)
        if match:
            return match.group(1).strip().replace(" ", "")
    return "Review Manually"
```

- **Szintaktika és Működés soronként:**
  - `def find_match(...)`: Egy belső segédfüggvény (*nested helper function*). Azért hoztuk létre a függvényen belül, hogy ne kelljen minden egyes mezőnél újra leírni a keresési és hibakezelési ciklust.
  - `for pattern in patterns:`: Végigmegy egy mintalistán prioritási sorrendben (első a legszigorúbb, utolsó a leglazább tartalékszabály).
  - `match = re.search(pattern, text_block, re.IGNORECASE)`:
    - `re.search`: A Python reguláris kifejezés könyvtára. Megkeresi az első olyan szövegrészt, ami megfelel a mintának.
    - `re.IGNORECASE`: Nem számít, hogy kis- vagy nagybetűvel van írva a szövegben.
  - `if match:`: Ha talált egyezést:
    - `match.group(1)`: **KRITIKUS!** A regex mintában kerek zárójelbe `(...)` tett értékes adatot emeli ki (tehát lehagyja a kulcsszót, pl. a `Shipment:` szócskát, és csak magát a számot adja vissza).
    - `.strip()`: Levágja a felesleges szóközöket az elejéről és a végéről.
    - `.replace(" ", "")`: Kitörli a számjegyek közé ékelődött belső szóközöket (pl. az OCR `2004 812 302`-ként olvasott számából `2004812302`-t csinál).
    - `return ...`: Azonnal visszaadja a megtisztított adatot és leállítja a további minták próbálgatását.
  - `return "Review Manually"`: Ha a lista egyik mintája sem talált semmit a szövegben, nem dob hibát, hanem beírja az ellenőrzést kérő jelölést.

---

3. Reguláris Kifejezések (Regex Minták) Részletesen

Itt definiáljuk a keresési mintákat. Nézzük meg a regex jelölések jelentését:

Regex Szimbólum Gyorskalauz:

- `\d`: Tetszőleges számjegy (0–9).
- `\d{4}`: Pontosan 4 darab számjegy egymás után.
- `\d{6,}`: Legalább 6 (vagy több) számjegy egymás után.
- `\s`: Bármilyen szóköz, tabulátor vagy sortörés.
- `\s*`: Nulla vagy több szóköz (lehet ott szóköz, de nem kötelező).
- `\s?`: Nulla vagy egy darab szóköz (opcionális szóköz).
- `+`: Az előtte lévő karakterből legalább 1 vagy több darab kötelező.
- `\b`: Szóhatár (*word boundary*). Biztosítja, hogy a szám ne egy hosszabb cikkszám közepe legyen, hanem önállóan álljon.
- `(?: ... )`: Nem elfogó csoport (csak csoportosít, de nem ezt adjuk vissza).
- `( ... )`: **Elfogó csoport** (*capture group 1*). **Ez maga a kinyerni kívánt érték!**

A) CMR Szám Minták

```
cmr_patterns = [r"N°\s*(\d{4}\s?/\s?\d+)", r"\b(\d{4}\s?/\s?\d+)\b"]
```

1. `r"N°\s*(\d{4}\s?/\s?\d+)"`:
   - Keresi a `N°` jelzést, utána tetszőleges szóközt (`\s*`).
   - `(\d{4}\s?/\s?\d+)`: 4 jegyű évszám (pl. `2026`), utána egy perjel `/` (szóközzel vagy anélkül), majd a sorszám (pl. `41041`).
2. `r"\b(\d{4}\s?/\s?\d+)\b"` (Tartalék):
   - Ha a `N°` jel lekopott vagy elmosódott a lap jobb felső sarkában, önmagában keres egy `ÉÉÉÉ/SZÁM` struktúrát.

B) Shipment Szám Minták

```
shipment_patterns = [
    r"Shipment\s*nr\.\s*:\s*(\d[\d\s+]+)",
    r"Shipment\s*:\s*(\d[\d\s+]+)",
    r"\b(2004\d{6,})\b",
]
```

1. `Shipment\s*nr\.\s*:\s*(\d[\d\s+]+)`: Keresi a `Shipment nr.:` feliratot, utána a számokat. A `[\d\s+]+` kezeli, ha az OCR hibásan `+` jelet vagy szóközt olvasott a számok közé.
2. `Shipment\s*:\s*(\d[\d\s+]+)`: Ugyanez a rövidebb `Shipment:` felirattal.
3. `\b(2004\d{6,})\b` (Tartalék): Ha egyáltalán nincs felirat a rubrikában, keres egy olyan önálló számot, ami `2004`-gyel kezdődik és legalább 6 további számjegy követi (összesen 10 jegyű Bosch fuvarszám).

C) Transport Order (TO) Minták

```
to_patterns = [r"T0[:!]*\s*([A-Z]{2}\d+)", r"\b(EL\d{10})\b"]
```

1. `T0[:!]*\s*([A-Z]{2}\d+)`:
   - `T0`: Mivel a függvény elején minden `O`-t `0`-ra cseréltünk, a `TO:` feliratból `T0:` lett!
   - `[:!]*`: Kezeli, ha a kettőspontot az OCR felkiáltójelnek `!` olvasta vagy összeragadt.
   - `([A-Z]{2}\d+)`: 2 betűs kód (pl. EL), amit számok követnek.
2. `\b(EL\d{10})\b` (Tartalék): Keres egy önállóan álló, `EL`-lel kezdődő 10 számjegyű azonosítót bárhol az 5-ös mezőben vagy a vámzár alatt.

D) Loading List (Box 5) Minta

```
loading_list_patterns = [r"Loading\s*list\s*[:\s]*([A-Z]{2}\d+[_A-Z0-9]+)"]
```

- `Loading\s*list\s*[:\s]*`: Keresi a `Loading list:` vagy `Loadinglist:` feliratot.
- `([A-Z]{2}\d+[_A-Z0-9]+)`:
  - `[A-Z]{2}\d+`: 2 betű és számok (pl. `HU03`).
  - `[_A-Z0-9]+`: Aláhúzás és a célország kódja a sorszámmal (pl. `_DE2604270001` vagy `_GR2606240001`).
  - **Eredmény:** Nem áll meg a `HU03`-nál, hanem a teljes `HU03_DE2604270001` azonosítót egyben emeli ki.

E) Vámzár és Dátum Minták

```
seal_patterns = [r"Vámzár\s*:\s*(\d+)", r"\b(00\d{4,6})\b"]
date_patterns = [r"(\d{4}-\d{2}-\d{2})", r"(\d{2}[-./]\d{2}[-./]\d{4})"]
```

- **Vámzár:** Keresi a `Vámzár:` kulcsszót, vagy tartalékként egy `00`-val kezdődő 6 jegyű sorszámot.
- **Dátum:** Kezeli az ISO formátumot (`2026-04-27`) és a pontozott/peres európai formátumot (`27.04.2026` vagy `27/04/2026`).

---

4. DALI Adatbázis-Összekapcsolás

```
shipment_num = find_match(shipment_patterns, text).replace("+", "")

# DALI keresés a Shipment alapján
invoice_from_dali = invoice_lookup.get(shipment_num, "Not Found in DALI")
```

- **Szintaktika:**
  - `find_match(shipment_patterns, text)`: Kinyeri a Shipment számot a fenti szabályokkal.
  - `.replace("+", "")`: Ha áthúzás miatt + került a számba, kitörli.
  - `invoice_lookup.get(kulcs, alapertelmezett_ertek)`:
    - A Python szótárak legbiztonságosabb keresőfüggvénye.
    - Megnézi a `podtabla.xlsx`-ből betöltött memóriatárban, hogy a `shipment_num` szerepel-e a `transport szam` oszlopban.
    - Ha megtalálja, azonnal visszaadja a hozzá tartozó számlaszámot (`szamladelivery.VBELN`).
    - Ha nincs a táblázatban, nem áll le hibával, hanem visszaadja a `"Not Found in DALI"` szöveget.
- **Logika:** Ezzel a két sorral kapcsoljuk össze a fizikai papír alapú OCR-t a vállalat SAP/DALI rendszerével.

---

5. Dátum Formázása és Egységesítése

```
date_str = find_match(date_patterns, text)
data["Dátum"] = "Review Manually"
if date_str != "Review Manually":
    try:
        parts = re.split(r"[-./]", date_str)
        if len(parts[0]) == 4:
            data["Dátum"] = f"{parts[0]}-{parts[1]}-{parts[2]}"
        else:
            data["Dátum"] = f"{parts[2]}-{parts[1]}-{parts[0]}"
    except Exception:
        pass
```

- **Szintaktika és Logika:**
  - `re.split(r"[-./]", date_str)`: Feldarabolja a dátumot a kötőjelek, pontok vagy perjelek mentén darabokra (`parts`).
  - `if len(parts[0]) == 4:`: Ha az első darab 4 karakter hosszú (pl. `2026`), akkor az év volt elöl (`YYYY-MM-DD`).

- `else:`: Ha az első darab 2 karakter (pl. 27), akkor európai formátum volt (`DD.MM.YYYY`), amit átforgatunk nemzetközi `YYYY-MM-DD` formátumra (`f"{parts[2]}-{parts[1]}-{parts[0]}"`).
- **Cél:** Az Excel táblázatban és a fájlnevekben minden dátum egységesen nézzen ki, megkönnyítve az időrendi rendezést.

---

6. Partner és Telephely Felismerése (Bélyegző Logika)

```
data["Címzett"], data["Átvevő Telephely"], data["Box 24 Bélyegző"] = ("Unknown", "Unknown", "No")
for name, info in KNOWN_CONSIGNEES.items():
    if any(re.search(kw, text, re.IGNORECASE) for kw in info["keywords"]):
        data.update({
            "Címzett": name,
            "Átvevő Telephely": info["site"],
            "Box 24 Bélyegző": "Yes",
        })
        break
```

- **Szintaktika:**
  - Alapértelmezésként beállítja az `"Unknown"` és `"No"` értékeket (ha semmit sem találunk).
  - `for name, info in KNOWN_CONSIGNEES.items():`: Végiglépked a partner-szótáron.
  - `any(...)`: Logikai VAGY vizsgálat. Ha a partnerhez megadott kulcsszavak (kw) közül **bármelyik** illeszkedik a szövegre:
    - `data.update(...)`: Beírja a hivatalos nevet, telephelyet, és a bélyegzőt `"Yes"`-re állítja.
    - `break`: Megállítja a ciklust, mert a partnert sikeresen azonosítottuk.

---

7. Hogyan tudsz manuálisan módosítani ezen a függvényen?

Ha a jövőben új CMR dokumentumot kapsz, a következőket teheted:

1. **Új mintát adhatsz hozzá:** Ha a CMR szám formátuma megváltozik (pl. megjelenik egy kötőjeles verzió: `2026-41041`), egyszerűen beszúrod a listába:

   ```
   cmr_patterns = [
       r"N°\s*(\d{4}\s?/\s?\d+)",
       r"\b(\d{4}-\d{5})\b",  # ÚJ MINTA HOZZÁADÁSA
       r"\b(\d{4}\s?/\s?\d+)\b",
   ]
   ```

2. **Karaktertisztítás bővítése:** Ha az OCR egy másik betűt téveszt el folyamatosan (pl. az `1`-est elválasztó vonalnak `|` látja), hozzáadod a fejléc alá:

   ```
   text = full_text.replace("O", "0").replace("|", "1")
   ```

3. **Új partner rögzítése:** Nem kell a függvény logikájához nyúlnod; elég a kód elején lévő `KNOWN_CONSIGNEES` szótárat bővíteni egy új sorral.